Blog Management System API

Entity Relationship Diagram Documentation

Author: Md Ismile Hosen Siam
Version: 2.0
Database: MongoDB

Date: 21 September, 2026

Contents

Introduction 1

Database Overview 1

Entity Relationship Diagram 2

Entity Summary 2

Entity Details 3

- **User Entity** 3
- **Session Entity** 3
- **OTP Entity** 4
- **PasswordResetToken Entity** 4
- **Category Entity** 5
- **Article Entity** 5
- **Comment Entity** 6
- **SystemInfo Entity** 6

Entity Relationships 7

Introduction

This Entity Relationship Diagram (ERD) Documentation explains the database structure of the Blog Management System API. It describes all database entities, their attributes, relationships, and constraints.

The Blog Management System API is a backend REST API that allows users to register, verify their email, log in, and manage blog content. It supports two user roles — registered users, who create, update, and delete their own articles, comment on articles, and reply to comments; and administrators, who manage users, categories, and system-wide content. Public visitors can browse published articles and active categories without an account.

This document helps developers, database designers, testers, and other team members understand how the database is designed and how different entities are connected. It can also be used as a reference for system development, testing, and future maintenance.

Database Overview

The Blog Management System database is designed to support the system's five functional modules: authentication and session management, user management, article management, category management, and comment management. It stores and manages data for user accounts, device sessions, one-time passcodes, password-reset tokens, categories, articles, comments and system-level setup state.

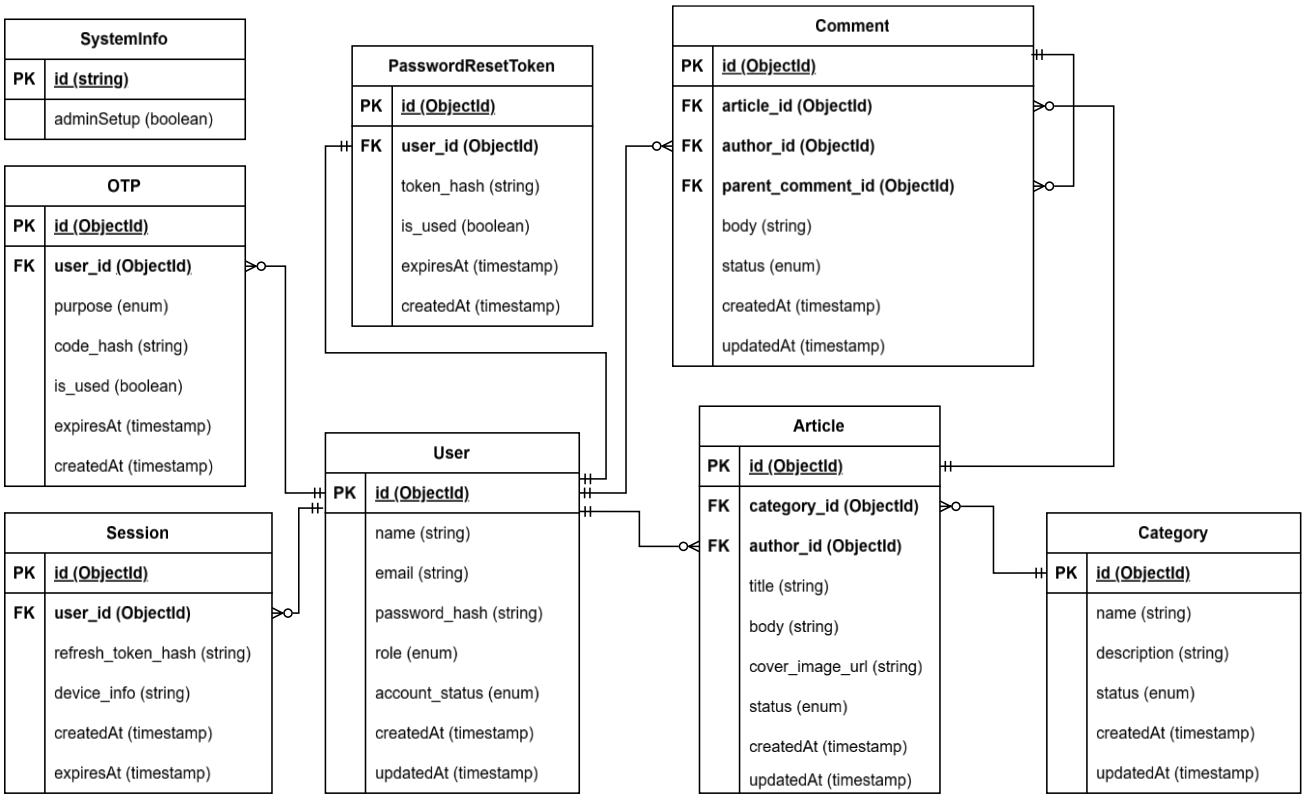
The database is implemented on MongoDB, a document-oriented, NoSQL database. This documentation nonetheless presents a logical/conceptual entity model — entities, keys, cardinalities, and constraints — because a technology-neutral logical model is the correct basis for schema design regardless of the underlying engine. Each logical entity below corresponds to one MongoDB collection, and every primary key is uniformly named id, mirroring MongoDB's own native _id primary-key field. The SystemInfo collection is the exception. Its id is a fixed string key, since it holds a single settings document. Foreign keys are implemented as ObjectId references rather than embedded sub-documents, since several entities must be independently paginated, filtered, and administered.

The database uses constraints such as unique keys, indexes, and enums to improve data integrity and performance. There are no soft-delete fields in this design — deletions in the system are hard deletes — with the sole exception that Category records are never deleted at all, only toggled between active and unavailable status.

In the current design, status and account-state changes are not individually attributed or timestamped at the schema level. Auditing needs are currently met through infrastructure-level application/API logs (Winston + ELK) rather than persisted per-record history

This database design provides a clear and structured way to manage blog content and account data, and supports the main workflows of the Blog Management System API.

Entity Relationship Diagram



Entity Summary

The Blog Management System database contains 8 entities. Each entity stores data for a specific part of the system.

Entity	Purpose
User	Stores account, credential, role, and account status for every registered user and administrator.
Session	Stores one authenticated device/browser login, supporting multi-device sign-in and per-device logout.
OTP	Stores one-time passcodes used for both email verification and password-reset-request verification.
PasswordResetToken	Stores the short-lived, single-use credential issued after OTP verification and required to reset a password.
Category	Stores article categories used for classification, filtering, search, and sort on article listings.
Article	Stores blog articles authored by a user or admin.
Comment	Stores comments and threaded replies on articles.
SystemInfo	Stores system-level state, such as whether the initial administrator setup is complete.

Entity Details

User Entity

Purpose

The User entity represents an account holder — either a standard user or an administrator. It stores personal details, login credentials, role, verification state, and account status.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier for the user.
name	STRING	—	Yes	—	User's display name.
email	STRING	—	Yes	—	User's login email address.
password_hash	STRING	—	Yes	—	bcrypt hash of the password.
role	ENUM	—	Yes	user	User's role in the system.
account_status	ENUM	—	Yes	pending	Current account status.
createdAt	TIMESTAMP	—	Yes	now()	Record creation time.
updatedAt	TIMESTAMP	—	Yes	now()	Last update time.

role: user, admin

account_status: pending, active, blocked

Unique Constraints: email

Indexes: email, role, account_status

Session Entity

Purpose

The Session entity represents one authenticated device/browser login. It backs the system's multi-device session model, enabling logout from the current device, logout from all devices, logout from a specific device, and listing of active sessions.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Identifies a session/device
user_id	ObjectId	User.id	Yes	—	Reference of user.
refresh_token_hash	STRING	—	Yes	—	Hashed refresh token.
device_info	STRING	—	Yes	—	Device information.
createdAt	TIMESTAMP	—	Yes	now()	Session issue time.
expiresAt	TIMESTAMP	—	Yes	—	Valid for 30 days.

Unique Constraints: refresh_token_hash

Indexes: user_id

OTP Entity

Purpose

The OTP entity stores a single, purpose-tagged one-time passcode used for both email verification and password-reset-request verification.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier of the OTP.
user_id	ObjectId	User.id	Yes	—	The account the OTP was issued for.
purpose	ENUM	—	Yes	—	Distinguishes the email-verification and password-reset flows.
code_hash	STRING	—	Yes	—	Hashed OTP code.
is_used	BOOLEAN	—	Yes	false	Indicates whether the OTP has already been consumed.
expiresAt	TIMESTAMP	—	Yes	—	Valid for 5 minutes.
createdAt	TIMESTAMP	—	Yes	now()	Issue time.

purpose: email_verification, password_reset

Unique Constraints: (user_id, purpose)

Indexes: (user_id, purpose)

PasswordResetToken Entity

Purpose

The PasswordResetToken entity stores the short-lived credential issued after OTP verification and required to actually change the password.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier of the reset token.
user_id	ObjectId	User.id	Yes	—	The account the token authorizes a reset for.
token_hash	STRING	—	Yes	—	Hashed representation of the token.
is_used	BOOLEAN	—	Yes	false	Indicates single-use consumption of the token.
expiresAt	TIMESTAMP	—	Yes	—	Valid for 5 minutes.
createdAt	TIMESTAMP	—	Yes	now()	Issue time.

Unique Constraints: token_hash

Indexes: user_id

Category Entity

Purpose

The Category entity represents an article category, managed by administrators, and used for classification, filtering, search, and sort on article listings.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier of the category.
name	STRING	—	Yes	—	Name of the category.
description	STRING	—	No	NULL	Description of the category.
status	ENUM	—	Yes	active	Current status of the category.
createdAt	TIMESTAMP	—	Yes	now()	Record creation time.
updatedAt	TIMESTAMP	—	Yes	now()	Last update time.

status: active, unavailable

Unique Constraints: name

Indexes: status, name

Article Entity

Purpose

The Article entity is the core content entity — a blog post authored by a user, optionally moderated into a published state by an administrator.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier of the article.
category_id	ObjectId	Category.id	Yes	—	Reference to the article's category.
author_id	ObjectId	User.id	Yes	—	Reference to the authoring user.
title	STRING	—	Yes	—	Title of the article.
body	STRING	—	Yes	—	Body of the article.
cover_image_url	STRING	—	Yes	—	URL of the cover image.
status	ENUM	—	Yes	published	Current status.
createdAt	TIMESTAMP	—	Yes	now()	Record creation time.
updatedAt	TIMESTAMP	—	Yes	now()	Last update time.

status: draft, published

Unique Constraints: none

Indexes: category_id, author_id, status, createdAt, title

Comment Entity

Purpose

The Comment entity stores a comment or threaded reply on an article. Top-level comments and replies are represented by the same entity, distinguished by a self-referencing parent link — a reply is simply a comment whose parent_comment_id is not null. When a comment is deleted, all of its direct and nested replies must also be deleted.

Attributes

Field	Type	FK	Required	Default	Description
id	ObjectId	—	Yes	auto()	Unique identifier of the comment.
article_id	ObjectId	Article.id	Yes	—	Reference of the article.
author_id	ObjectId	User.id	Yes	—	Reference of the owner.
parent_comment_id	ObjectId	Comment.id	No	NULL	Reference to the parent comment.
body	STRING	—	Yes	—	Comment text.
status	ENUM	—	Yes	public	Current status
createdAt	TIMESTAMP	—	Yes	now()	Record creation time.
updatedAt	TIMESTAMP	—	Yes	now()	Last update time.

status: public, hidden

Unique Constraints: none

Indexes: article_id, author_id, parent_comment_id, status

SystemInfo Entity

Purpose

The SystemInfo entity stores global system state that doesn't belong to any user or content record. It currently tracks whether the initial administrator setup has been completed.

Attributes

Field	Type	FK	Required	Default	Description
id	STRING	—	Yes	—	Fixed identifier of the system-info document.
adminSetup	BOOLEAN	—	Yes	false	Indicates whether the initial admin account has been set up.

Unique Constraints: none

Indexes: none

Entity Relationships

Parent Entity	Child Entity	Relationship	Foreign Key
User	Session	One-to-Many (1: N)	Session.user_id
User	OTP	One-to-Many (1: N)	OTP.user_id
User	PasswordResetToken	One-to-One (1: 1)	PasswordResetToken.user_id
User	Article	One-to-Many (1: N)	Article.author_id
User	Comment	One-to-Many (1: N)	Comment.author_id
Category	Article	One-to-Many (1: N)	Article.category_id
Article	Comment	One-to-Many (1: N)	Comment.article_id
Comment	Comment	One-to-Many (1: N)	Comment.parent_comment_id